Pipeline de implementación de un esquema de conocimiento cero

Del protocolo de Schnorr a los SNARKs: algoritmos, librerías en Python y aplicaciones

José de Jesús Angel Angel
Facultad de Ingeniería, Universidad Anáhuac México · Lab-IA

27 de octubre de 2026

Resumen

Una prueba de conocimiento cero permite que un probador convenza a un verificador de que una afirmación es verdadera, o de que conoce un secreto, sin revelar nada más. Este documento describe el pipeline completo para implementarla: especificación de la relación, aritmetización, elección del sistema de pruebas, setup, generación y verificación. Se presentan en pseudocódigo el protocolo Σ de Schnorr, su versión no interactiva por Fiat–Shamir, la construcción de Goldreich, Micali y Wigderson que obtiene conocimiento cero para todo NP a partir de la 3-coloración de grafos, y el flujo general de un zk-SNARK; se incluye una implementación de referencia en Python, una revisión de las librerías disponibles y un catálogo de aplicaciones reales y escenarios de uso.

Índice general

1 Panorama del pipeline **3**

2 Esquema base: el protocolo Σ de Schnorr **4**
2.1 Versión interactiva . . . 4
2.2 Por qué es una prueba de conocimiento cero . . . 5
2.3 Versión no interactiva (Fiat–Shamir) . . . 5
2.4 Variantes que salen de la misma plantilla . . . 6

3 Conocimiento cero para todo NP: Goldreich, Micali y Wigderson **6**
3.1 Por qué un problema NP-completo . . . 7
3.2 Ingrediente criptográfico: esquemas de compromiso . . . 7
3.3 El protocolo . . . 8
3.4 Análisis de seguridad . . . 8
3.5 Del problema NP al esquema: pipeline . . . 10
3.6 Implementación en Python . . . 10

4 Pipeline general con SNARKs **12**
4.1 Etapa 1 · Especificación . . . 12
4.2 Etapa 2 · Aritmetización . . . 12
4.3 Etapa 3 · Elección del sistema . . . 13
4.4 Etapas 4 a 7 · Setup, testigo, prueba y verificación . . . 13
4.5 Etapa 8 · Integración y auditoría . . . 14

5 Librerías en Python y herramientas del ecosistema **14**
5.1 Librerías de Python 14
5.2 Herramientas de producción (fuera de Python) 15

6 Implementación de referencia en Python **15**

7 Aplicaciones reales en producción **18**

8 Escenarios donde el conocimiento cero puede usarse **18**

1 Panorama del pipeline

Implementar conocimiento cero es recorrer siempre el mismo camino: fijar la afirmación, escribirla como ecuaciones sobre un campo finito, elegir el sistema de pruebas, generar la prueba con el testigo secreto y verificarla sin ese testigo. Lo que cambia entre esquemas es cuánto cuesta cada etapa y qué supuestos de seguridad se aceptan.

Terminología

- **Afirmación** x: lo público. Ejemplo: «conozco el logaritmo discreto de Y».
- **Testigo** w: el secreto que hace verdadera la afirmación. Nunca sale del probador.
- **Relación** $R(x, w)$: el predicado que el verificador comprueba sin ver w. El lenguaje asociado es $L_R = \{x : \exists\, w,\ (x, w) \in R\}$.
- **Probador** P y **verificador** V: Peggy y Víctor. P conoce w; V sólo conoce x.
- **Parámetros públicos** pp: grupo, curva, campo, claves de prueba pk y de verificación vk.

Un sistema de pruebas de conocimiento cero para R debe cumplir tres propiedades:

$$\text{Completitud:}\quad (x, w) \in R \implies \Pr\big[\langle \mathsf{P}(w), \mathsf{V}\rangle(x) = 1\big] = 1, \tag{1}$$

$$\text{Solidez:}\quad x \notin L_R \implies \Pr\big[\langle \mathsf{P}^*, \mathsf{V}\rangle(x) = 1\big] \le \varepsilon, \tag{2}$$

$$\text{Conocimiento cero:}\quad \exists\, \mathsf{Sim} :\ \mathsf{Sim}(x) \approx \mathrm{View}_{\mathsf{V}}\big[\langle \mathsf{P}(w), \mathsf{V}\rangle(x)\big]. \tag{3}$$

Hay dos familias de implementación. Los **protocolos** Σ (Schnorr, Chaum–Pedersen, pruebas OR) sirven para afirmaciones algebraicas concretas y se escriben a mano en pocas líneas. Los **sistemas de propósito general** (Groth16, PLONK, STARKs, Bulletproofs) prueban cualquier cómputo, a cambio de compilarlo a un circuito. La figura 1 resume ambas rutas.

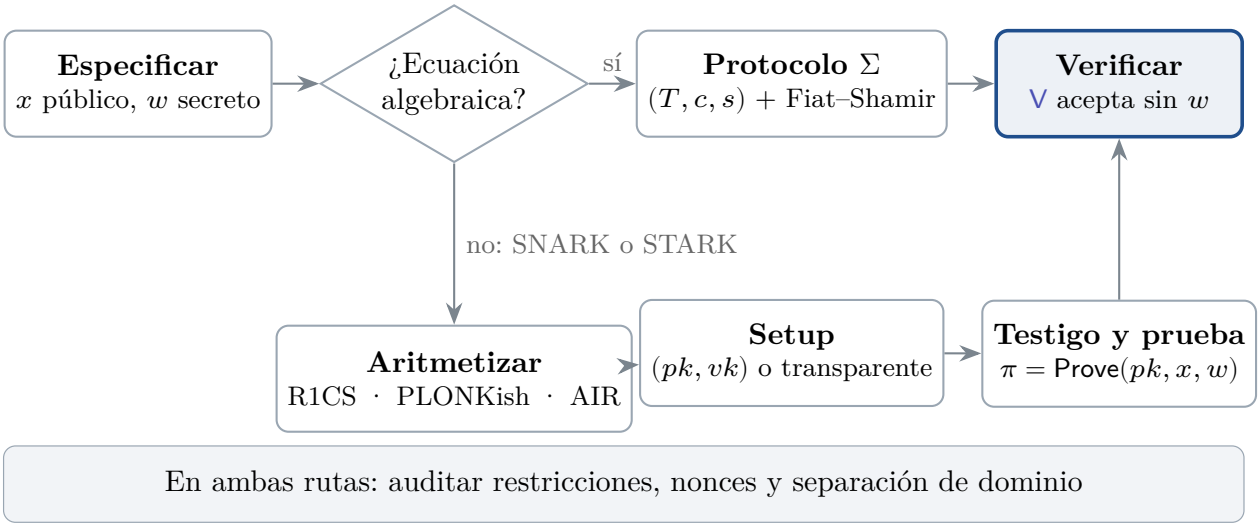


Figura 1: Pipeline de conocimiento cero. Una afirmación algebraica sencilla va directo a un protocolo Σ; cualquier otro cómputo pasa por circuito, setup y probador.

Etapas del pipeline general.

1. **Especificar** la afirmación pública x, el testigo w y la relación R.
2. **Aritmetizar**: traducir R a restricciones sobre un campo finito (R1CS, PLONKish o AIR).
3. **Elegir el sistema de pruebas** según tamaño de prueba, costo de verificación y tipo de setup.
4. **Setup**: generar parámetros públicos; en Groth16 una ceremonia por circuito, en PLONK una universal, en STARKs y Bulletproofs ninguna.
5. **Calcular el testigo** completo (todas las señales intermedias del circuito) a partir de w.
6. **Probar**: P produce $\pi = \mathsf{Prove}(pk, x, w)$.
7. **Verificar**: V acepta si $\mathsf{Verify}(vk, x, \pi) = 1$, en local o en un contrato inteligente.
8. **Integrar y auditar**: serialización, aleatoriedad segura, revisión del circuito y pruebas con transcripciones mal formadas.

2 Esquema base: el protocolo Σ de Schnorr

Schnorr [7] prueba conocimiento de x tal que $Y = x \cdot G$ en un grupo de orden primo q, con tres mensajes y sin revelar x. Es la plantilla de todos los protocolos Σ: compromiso, reto, respuesta.

Parámetros y relación

Sea $\mathbb{G}$ un grupo cíclico de orden primo q con generador G (por ejemplo, la curva secp256k1 o Ristretto255) y H una función hash para la versión no interactiva. La relación es

$$R_{\mathrm{DL}} = \{(Y, x) \in \mathbb{G} \times \mathbb{Z}_q : Y = x \cdot G\}.$$

2.1 Versión interactiva

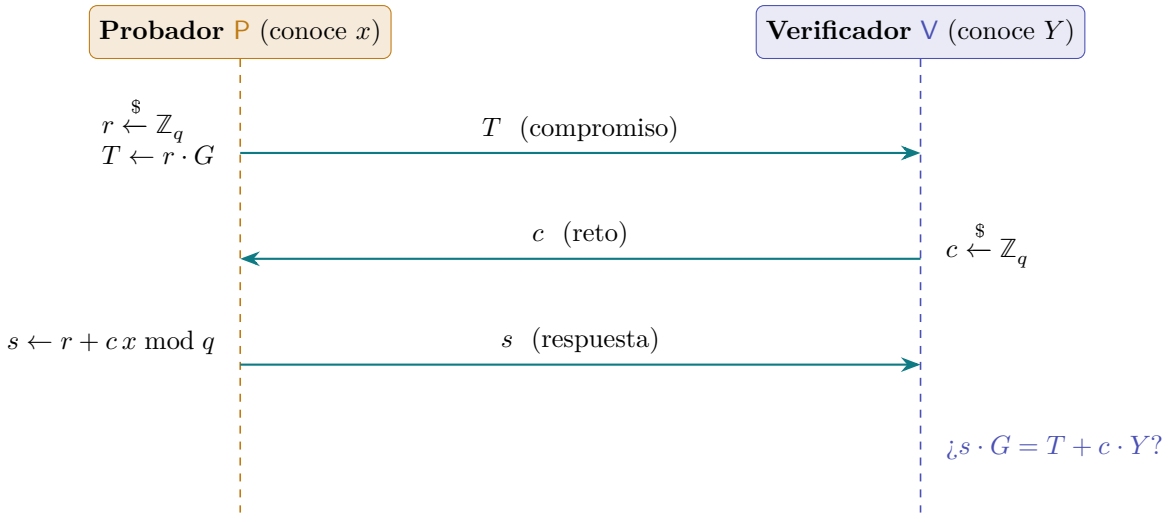


Figura 2: Diagrama de mensajes del protocolo de Schnorr.

Algoritmo 1 Schnorr interactivo

Entrada: públicas $\mathbb{G}, q, G, Y$; privada x (sólo P)

1: P: $r \xleftarrow{\$} \mathbb{Z}_q$ ▷ nonce nuevo en cada prueba
2: P: $T \leftarrow r \cdot G$
3: P $\longrightarrow$ V: T ▷ compromiso
4: V: $c \xleftarrow{\$} \mathbb{Z}_q$
5: V $\longrightarrow$ P: c ▷ reto
6: P: $s \leftarrow (r + c\,x) \bmod q$
7: P $\longrightarrow$ V: s ▷ respuesta
8: **si** $s \cdot G = T + c \cdot Y$ **entonces**
9: **devolver** acepta
10: **si no**
11: **devolver** rechaza
12: **fin si**

La verificación funciona porque

$$s \cdot G = (r + c\,x) \cdot G = r \cdot G + c\,(x \cdot G) = T + c \cdot Y. \tag{4}$$

2.2 Por qué es una prueba de conocimiento cero

Proposición 1 (Solidez especial)**.** *Si* (T, c_1, s_1) *y* (T, c_2, s_2) *son transcripciones aceptadas con* $c_1 \neq c_2$*, entonces*

$$x = (s_1 - s_2)\,(c_1 - c_2)^{-1} \bmod q. \tag{5}$$

Prueba. De $s_i \cdot G = T + c_i \cdot Y$ se obtiene $(s_1 - s_2) \cdot G = (c_1 - c_2) \cdot Y = (c_1 - c_2)x \cdot G$, y como G tiene orden primo q se despeja x. □

Quien responde bien a dos retos distintos con el mismo compromiso conoce x. Para el conocimiento cero frente a un verificador honesto basta un simulador que no use x:

Algoritmo 2 Simulador de Schnorr

Entrada: Y (sin x)

1: $c \xleftarrow{\$} \mathbb{Z}_q$, $s \xleftarrow{\$} \mathbb{Z}_q$
2: $T \leftarrow s \cdot G - c \cdot Y$
3: **devolver** (T, c, s) ▷ verifica y tiene la misma distribución que una real

En una ejecución real s es uniforme porque r lo es, y T queda determinado por (c, s); el simulador produce exactamente esa distribución.

2.3 Versión no interactiva (Fiat–Shamir)

Fiat y Shamir [2] reemplazan el reto por un hash de todo el contexto público. La prueba se convierte en un par (T, s) que cualquiera verifica sin conversar con P:

$$c = \mathsf{H}(\texttt{dominio} \,\|\, G \,\|\, Y \,\|\, T \,\|\, m) \bmod q. \tag{6}$$

Algoritmo 3 Schnorr no interactivo

función Prove-NI(x, Y, m)
  $r \xleftarrow{\$} \mathbb{Z}_q$ ▷ o determinista y con sal, RFC 6979 / BIP-340
  $T \leftarrow r \cdot G$
  $c \leftarrow \mathsf{H}(\texttt{dominio} \,\|\, G \,\|\, Y \,\|\, T \,\|\, m) \bmod q$
  $s \leftarrow (r + c\,x) \bmod q$
  devolver $\pi = (T, s)$
fin función

función Verify-NI$(Y, m, \pi = (T, s))$
  si $Y \notin \mathbb{G}$ **o** $T \notin \mathbb{G}$ **o** $Y = \mathcal{O}$ **entonces**
    devolver rechaza
  fin si
  $c \leftarrow \mathsf{H}(\texttt{dominio} \,\|\, G \,\|\, Y \,\|\, T \,\|\, m) \bmod q$
  devolver $[\, s \cdot G = T + c \cdot Y \,]$
fin función

Precaución

- Meter en el hash **todo** lo público (G, Y, T, el mensaje y una etiqueta de dominio). Omitir Y o T da lugar al ataque de «Fiat–Shamir débil».
- Nunca reutilizar r: con dos pruebas del mismo r se aplica (5) y se obtiene x. Es el error que expuso la clave de firma de la PlayStation 3 en ECDSA.
- Validar que los puntos recibidos pertenecen al subgrupo de orden primo.

2.4 Variantes que salen de la misma plantilla

Protocolo	Prueba que...	Uso típico
Schnorr	conozco x con $Y = x \cdot G$	autenticación, firmas Schnorr y EdDSA
Chaum–Pedersen	$\log_G A = \log_H B$	descifrado verificable, VRF, votación
Okamoto / Pedersen	conozco (a, b) con $C = a \cdot G + b \cdot H$	apertura de compromisos
Pruebas OR (CDS) [8]	conozco el secreto de Y_1 **o** de Y_2	anonimato en anillos, votos 0/1
Pruebas AND	conozco ambos secretos	credenciales con varios atributos

Tabla 1: Protocolos Σ derivados de Schnorr.

3 Conocimiento cero para todo NP: Goldreich, Micali y Wigderson

Schnorr resuelve una sola relación algebraica. Goldreich, Micali y Wigderson [3] mostraron en 1986 que basta una prueba de conocimiento cero para *un* problema NP-completo para obtenerla para *todo* NP: se reduce la afirmación a una instancia de 3-coloración de grafos y se prueba que se conoce una coloración válida, revelando en cada ronda sólo los colores de una arista. Es el resultado

que convirtió el conocimiento cero en una herramienta general y la base conceptual de los SNARKs de la sección 4.

3.1 Por qué un problema NP-completo

Lenguajes NP y reducciones

Un lenguaje L está en NP si existe una relación R_L verificable en tiempo polinomial tal que $x \in L \iff \exists\, w,\ |w| \le \mathrm{poly}(|x|),\ (x,w) \in R_L$. Una **reducción de Karp** [4] es una función f calculable en tiempo polinomial con $x \in L \iff f(x) \in L'$. Es **preservadora de testigos** (de Levin) si además existe g polinomial que transforma un testigo w de x en un testigo $g(x,w)$ de $f(x)$.

El problema 3-Col pregunta si un grafo $\mathcal{G} = (V, E)$ admite $\varphi : V \to \{1,2,3\}$ con $\varphi(u) \neq \varphi(v)$ para toda arista $(u,v) \in E$. Es NP-completo, así que cualquier afirmación NP (un circuito satisfacible, un Sudoku resuelto, un factor de n, una preimagen de un hash) se traduce en un grafo y su testigo en una coloración. La figura 3 resume esta idea.

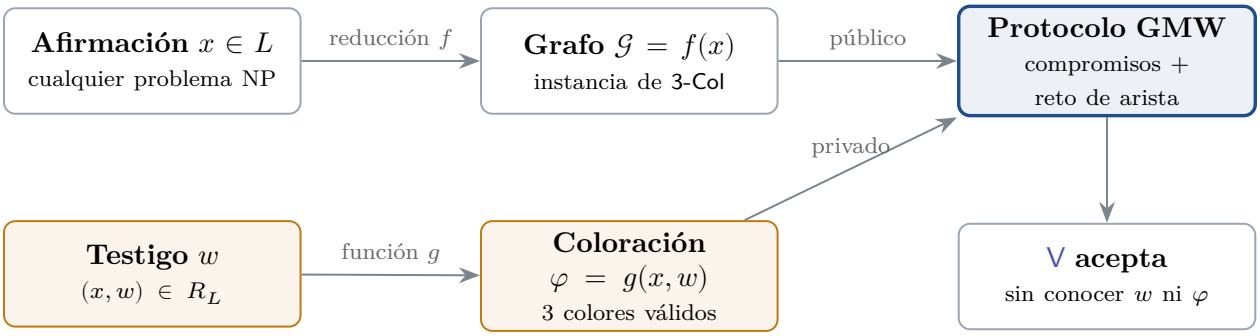


Figura 3: Cómo una reducción a 3-Col convierte cualquier afirmación NP en una prueba de conocimiento cero.

Proposición 2 (Goldreich–Micali–Wigderson, 1986)**.** *Si existen funciones unidireccionales, todo lenguaje en NP tiene una prueba interactiva de conocimiento cero computacional.*

Las funciones unidireccionales bastan para construir esquemas de compromiso, que son el único ingrediente criptográfico del protocolo.

3.2 Ingrediente criptográfico: esquemas de compromiso

Compromiso

Un esquema de compromiso es un par $(\mathsf{Commit}, \mathsf{Open})$: $C = \mathsf{Commit}(m; \rho)$ con aleatoriedad ρ, y $\mathsf{Open}(C, m, \rho) \in \{0,1\}$. Debe ser

- **ocultador**: C no revela información sobre m (es un sobre cerrado);
- **vinculante**: es inviable abrir C como dos valores distintos $m \neq m'$.

Dos instancias habituales:

$$\text{Por hash:} \quad C = \mathsf{H}(m \,\|\, \rho), \qquad \rho \xleftarrow{\$} \{0,1\}^{256}, \tag{7}$$

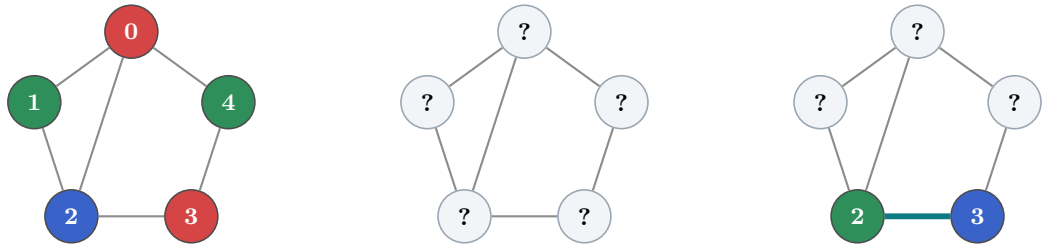
$$\text{Pedersen:} \quad C = m \cdot G + \rho \cdot H, \qquad \rho \xleftarrow{\$} \mathbb{Z}_q,\ \log_G H \text{ desconocido}. \tag{8}$$

Nota

Pedersen es perfectamente ocultador y computacionalmente vinculante: con él, GMW se vuelve un *argumento* (sólido frente a probadores de cómputo acotado) con conocimiento cero perfecto. Con un compromiso perfectamente vinculante (por ejemplo, ElGamal) se obtiene una *prueba* con conocimiento cero computacional, como en el enunciado original. Ambas propiedades no pueden ser perfectas a la vez.

3.3 El protocolo

En cada ronda P baraja sus tres colores con una permutación aleatoria $\pi \in S_3$, se compromete con el color de cada vértice y abre sólo los dos extremos de la arista que V elige. La figura 4 muestra una ronda sobre un grafo de 5 vértices.



(a) Coloración secreta φ sólo la conoce P

(b) $\pi \circ \varphi$ en sobres $C_0, \ldots, C_4$ π: rojo→azul, verde→rojo, azul→verde

(c) V pide la arista $(2,3)$ abre C_2, C_3: verde $\neq$ azul

Figura 4: Una ronda del protocolo GMW. Como π es nueva en cada ronda, los dos colores revelados son un par aleatorio de colores distintos y no dicen nada de φ.

El protocolo completo repite la ronda k veces de forma secuencial, con π y ρ_i nuevos cada vez, y V acepta sólo si acepta todas.

3.4 Análisis de seguridad

Completitud. Si φ es válida, $\pi \circ \varphi$ también lo es, así que los dos colores abiertos siempre difieren.

Solidez. Si $\mathcal{G}$ no es 3-coloreable, cualquier asignación comprometida tiene al menos una arista monocromática (o un valor fuera de $\{1,2,3\}$). Por la propiedad vinculante, P^* no puede cambiar los colores tras ver el reto, así que V la descubre con probabilidad al menos $1/|E|$ por ronda:

$$\Pr[\text{engaño en } k \text{ rondas}] \le \left(1 - \frac{1}{|E|}\right)^k \le e^{-k/|E|}, \qquad k = \left\lceil |E| \, \ln \frac{1}{\varepsilon} \right\rceil \ \Rightarrow\ \text{error} \le \varepsilon. \tag{9}$$

Para el grafo de 14 aristas de la página interactiva de laboratorios y $\varepsilon = 10^{-6}$ bastan $k = 187$ rondas.

Algoritmo 4 Una ronda de GMW para 3-Col

Entrada: público $\mathcal{G} = (V, E)$ con $|V| = n$; privado $\varphi : V \to \{1, 2, 3\}$ válida (sólo P)

P: $\pi \xleftarrow{\$} S_3$ ▷ permutación nueva en cada ronda
para $i = 1, \dots, n$ **hacer**
  P: $\psi_i \leftarrow \pi(\varphi(i))$, $\rho_i \xleftarrow{\$} \{0,1\}^{256}$, $C_i \leftarrow \mathsf{Commit}(\psi_i; \rho_i)$
fin para
P $\longrightarrow$ V: $C_1, \dots, C_n$ ▷ compromiso
V: $(u, v) \xleftarrow{\$} E$
V $\longrightarrow$ P: (u, v) ▷ reto
P $\longrightarrow$ V: $(\psi_u, \rho_u), (\psi_v, \rho_v)$ ▷ respuesta: abre dos sobres
si $\mathsf{Open}(C_u, \psi_u, \rho_u) \wedge \mathsf{Open}(C_v, \psi_v, \rho_v) \wedge \psi_u \neq \psi_v \wedge \psi_u, \psi_v \in \{1, 2, 3\}$ **entonces**
  devolver acepta la ronda
si no
  devolver rechaza
fin si

Conocimiento cero. Un simulador sin φ adivina la arista que pedirá el verificador y prepara compromisos coherentes sólo con ella:

Algoritmo 5 Simulador de GMW (con rebobinado)

Entrada: $\mathcal{G} = (V, E)$ y acceso al verificador V^* (sin coloración)

repetir
  $(u', v') \xleftarrow{\$} E$; $(\psi_{u'}, \psi_{v'}) \xleftarrow{\$} \{(a, b) \in \{1, 2, 3\}^2 : a \neq b\}$
  $\psi_i \xleftarrow{\$} \{1, 2, 3\}$ para $i \notin \{u', v'\}$ ▷ basura que nunca se abrirá
  $C_i \leftarrow \mathsf{Commit}(\psi_i; \rho_i)$ para todo i
  $(u, v) \leftarrow \mathsf{V}^*(C_1, \dots, C_n)$
  si $(u, v) = (u', v')$ **entonces**
    devolver $(C_1, \dots, C_n; (u, v); (\psi_u, \rho_u), (\psi_v, \rho_v))$
  fin si
  rebobinar V^* a su estado inicial
fin repetir

Como los compromisos son ocultadores, V^* no distingue dónde está la arista preparada, acierta con probabilidad cercana a $1/|E|$ y el simulador termina en un número esperado de $|E|$ intentos por ronda. La transcripción simulada es indistinguible de la real: en ambas se ven n sobres cerrados y dos colores distintos uniformes.

Precaución

El conocimiento cero se preserva al repetir las rondas en **secuencia**. Ejecutarlas en **paralelo** reduce la latencia pero, en general, no preserva el conocimiento cero [6]; la versión no interactiva se obtiene con Fiat–Shamir sobre todas las rondas.

Algoritmo 6 Conocimiento cero para un lenguaje $L \in$ NP vía 3-Col

Entrada: afirmación x (pública), testigo w con $(x, w) \in R_L$ (sólo P), error objetivo ε

Ambos calculan $\mathcal{G} \leftarrow f(x)$ ▷ reducción de Karp pública, p. ej. circuito → SAT → 3-Col
P calcula $\varphi \leftarrow g(x, w)$ y comprueba que es válida
$k \leftarrow \lceil |E| \ln(1/\varepsilon) \rceil$
para $j = 1, \ldots, k$ **hacer**
    ejecutar el algoritmo 4 con $(\mathcal{G}, \varphi)$
    si la ronda j rechaza **entonces**
        devolver rechaza
    fin si
fin para
devolver acepta

3.5 Del problema NP al esquema: pipeline

Ejemplo de reducción (SAT → 3-Col). Se crea un triángulo de paleta con vértices V, F y Base; cada variable x_i da dos vértices $x_i, \bar{x}_i$ unidos entre sí y a Base, de modo que uno toma el color de V y el otro el de F; cada cláusula se codifica con un gadget OR de seis vértices que sólo es 3-coloreable si algún literal tiene el color de V. Una asignación que satisface la fórmula (el testigo w) se transforma directamente en la coloración φ. El tamaño crece de forma lineal en el número de cláusulas.

	GMW (3-Col)	**Blum (ciclo hamiltoniano)** [5]	**SNARK (R1CS)**
Problema NP-completo	3-coloración	ciclo hamiltoniano	satisfacibilidad de R1CS
Error por ronda	$1 - 1/\lvert E\rvert$	$1/2$	$\approx \deg / \lvert\mathbb{F}\rvert$ en una sola prueba
Compromisos	un sobre por vértice	matriz de adyacencia permutada	compromisos polinomiales (KZG, FRI)
Rondas	$\sim \lvert E\rvert \ln(1/\varepsilon)$	$\sim \log_2(1/\varepsilon)$	1 (no interactivo)
Tamaño de la prueba	lineal en $\lvert E\rvert$ por ronda	cuadrático en $\lvert V\rvert$ por ronda	constante o polilogarítmico
Uso	teórico y docente	teórico y docente	producción

Tabla 2: Tres construcciones basadas en problemas NP-completos.

Nota

Los SNARKs de la sección 4 siguen la misma estrategia que GMW: reducen la afirmación a un problema NP-completo (satisfacibilidad de R1CS o de un sistema PLONKish) y prueban que se conoce un testigo. Lo que cambia es el mecanismo de compromiso: en lugar de abrir dos vértices por ronda, se compromete el testigo completo como polinomio y se comprueba en un punto aleatorio, lo que reduce miles de rondas a una prueba de unos cientos de bytes.

3.6 Implementación en Python

El programa siguiente implementa el algoritmo 4 con compromisos por hash, el protocolo completo con k rondas y el simulador sin testigo, sobre el mismo grafo de 7 vértices y 14 aristas de la

página interactiva de laboratorios. Fue ejecutado con Python 3.

```python
"""Prueba de conocimiento cero de 3-coloración (Goldreich-Micali-Wigderson).
Compromisos por hash con nonce de 256 bits (ocultamiento y vinculación heurísticos)."""
import hashlib, secrets, random, math

def comprometer(valor: int):
    nonce = secrets.token_bytes(32)
    return hashlib.sha256(bytes([valor]) + nonce).hexdigest(), nonce

def abre(comp, valor, nonce):
    return hashlib.sha256(bytes([valor]) + nonce).hexdigest() == comp

class Probador:
    def __init__(self, coloracion):
        self.col = coloracion                       # testigo: lista de colores 0,1,2
    def compromisos(self):
        pi = [0, 1, 2]; random.SystemRandom().shuffle(pi)
        self.perm = [pi[c] for c in self.col]       # colores permutados
        pares = [comprometer(c) for c in self.perm]
        self.nonces = [n for _, n in pares]
        return [c for c, _ in pares]
    def abrir(self, u, v):
        return (self.perm[u], self.nonces[u]), (self.perm[v], self.nonces[v])

def ronda(prob, aristas):
    coms = prob.compromisos()                       # 1. P se compromete
    u, v = aristas[secrets.randbelow(len(aristas))]  # 2. V elige una arista
    (cu, nu), (cv, nv) = prob.abrir(u, v)           # 3. P abre dos sobres
    return abre(coms[u], cu, nu) and abre(coms[v], cv, nv) and cu != cv  # 4. V verifica

def simulador(n, aristas):
    """Sin testigo: adivina la arista, pone dos colores distintos ahí y basura en lo demás."""
    while True:
        u, v = aristas[secrets.randbelow(len(aristas))]
        cu, cv = random.SystemRandom().sample([0, 1, 2], 2)
        cols = [secrets.randbelow(3) for _ in range(n)]; cols[u], cols[v] = cu, cv
        coms = [comprometer(c) for c in cols]
        e = aristas[secrets.randbelow(len(aristas))]   # reto del verificador
        if e == (u, v):                                # si acertó, la transcripción sirve
            return [c for c, _ in coms], e, (cu, coms[u][1]), (cv, coms[v][1])

if __name__ == "__main__":
    aristas = [(0,1),(1,2),(2,3),(3,4),(4,5),(5,6),(6,0),(0,2),(1,3),(2,4),(3,5),(4,0),(5,1),(2,6)]
    valida = [0, 1, 2, 0, 1, 2, 1]
    falsa  = [0, 1, 2, 0, 1, 2, 0]                  # la arista (6,0) queda monocromática
    assert all(valida[a] != valida[b] for a, b in aristas)
    m = len(aristas)
    k = math.ceil(math.log(1e-6) / math.log(1 - 1/m))   # rondas para error <= 1e-6
    print("aristas:", m, " rondas para error <= 1e-6:", k)
    print("honesta aceptada:", all(ronda(Probador(valida), aristas) for _ in range(k)))
    print("tramposa aceptada:", all(ronda(Probador(falsa), aristas) for _ in range(k)))
```

```
    coms, e, (cu, _), (cv, _) = simulador(7, aristas)
    print("simulador: arista", e, "colores", cu, cv, "distintos:", cu != cv)
```

Salida al ejecutarlo:

```
aristas: 14  rondas para error <= 1e-6: 187
honesta aceptada: True
tramposa aceptada: False
simulador: arista (0, 1) colores 0 1 distintos: True
```

La probadora tramposa tiene una sola arista mala entre 14; aun así, en 187 rondas la probabilidad de que nunca la pidan es $(13/14)^{187} \approx 10^{-6}$.

4 Pipeline general con SNARKs

Para afirmaciones que no son una ecuación algebraica simple (un hash, una firma, una regla de negocio) se compila el cómputo a un circuito aritmético y se usa un sistema de propósito general. Ejemplo de trabajo: probar que conozco w tal que Poseidon$(w) = h$ y $w < 2^{64}$, revelando sólo h. La figura 5 muestra las transformaciones internas.

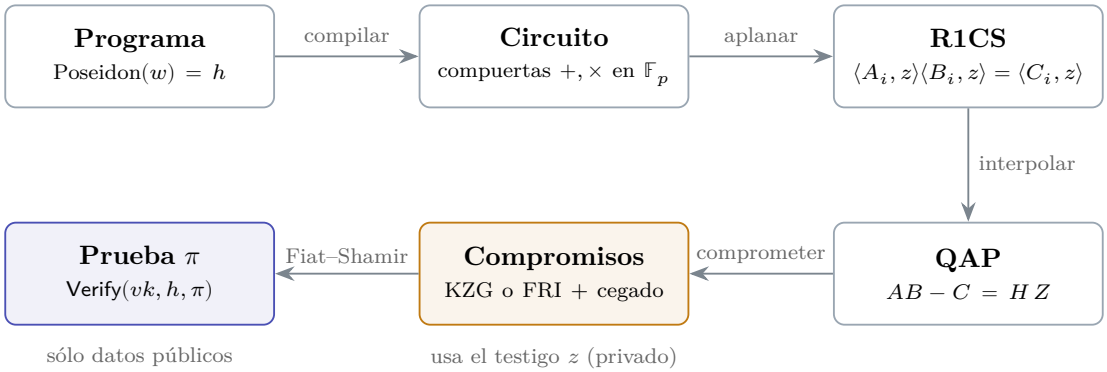


Figura 5: Transformaciones internas de un zk-SNARK tipo Groth16.

4.1 Etapa 1 · Especificación

```
AFIRMACIÓN pública:  h
TESTIGO privado:     w
RELACIÓN R(h, w):    Poseidon(w) == h   Y   0 <= w < 2^64
```

Se elige un hash amigable con circuitos (Poseidon, Rescue, MiMC) porque SHA-256 cuesta unas 25 000 a 30 000 restricciones R1CS por bloque, contra unos cientos para Poseidon.

4.2 Etapa 2 · Aritmetización

El cómputo se vuelve un sistema de restricciones sobre un campo primo $\mathbb{F}_p$ (para BN254, $p \approx 2^{254}$):

- **R1CS** (Groth16): con $z = (1, \text{públicos}, \text{privados}, \text{intermedios})$,

$$\langle A_i, z\rangle \cdot \langle B_i, z\rangle = \langle C_i, z\rangle, \qquad i = 1, \ldots, m. \tag{10}$$

- **PLONKish** (PLONK, Halo2): compuertas con selectores, restricciones de copia y tablas de búsqueda,
$$q_L\, a + q_R\, b + q_M\, a\, b + q_O\, c + q_C = 0. \tag{11}$$
- **AIR** (STARKs): una traza de ejecución con restricciones de transición entre filas consecutivas.

El rango $w < 2^{64}$ se impone descomponiendo w en bits:

$$b_i\,(b_i - 1) = 0 \quad (i = 0, \ldots, 63), \qquad \sum_{i=0}^{63} b_i\, 2^i = w. \tag{12}$$

```
CIRCUITO PreimagenAcotada (estilo Circom)
  señal privada  w
  señal pública  h
  bits[64] <- Num2Bits(64)(w)     // 64 restricciones booleanas + 1 de suma
  out      <- Poseidon(1)(w)
  restringir  out === h
```

4.3 Etapa 3 · Elección del sistema

Sistema	Tamaño de prueba	Verificación	Setup	Supuesto
Groth16 [9]	∼128–192 B (3 puntos)	constante	confiable, por circuito	emparejamientos, grupo genérico
PLONK (KZG) [10]	∼0.5–1 KB	constante	universal, actualizable	emparejamientos + KZG
Halo2 (IPA)	∼KB	lineal	ninguno	logaritmo discreto
STARK (FRI) [11]	decenas a cientos de KB	polilogarítmica	ninguno (transparente)	sólo hashes; poscuántico
Bulletproofs [12]	∼700 B (rango de 64 bits)	lineal	ninguno	logaritmo discreto

Tabla 3: Comparación de sistemas de pruebas de propósito general.

Nota

Verificar en cadena con gas mínimo ⇒ Groth16 o PLONK. Sin ceremonia y con seguridad poscuántica ⇒ STARK. Pruebas de rango cortas sin setup ⇒ Bulletproofs.

4.4 Etapas 4 a 7 · Setup, testigo, prueba y verificación

En Groth16 la verificación es una ecuación de emparejamientos:

$$e(A, B) = e(\alpha, \beta) \cdot e\Big(\textstyle\sum_i h_i\, \mathrm{IC}_i,\ \gamma\Big) \cdot e(C, \delta). \tag{13}$$

La idea central es que la identidad polinomial

$$A(X)\, B(X) - C(X) = H(X)\, Z(X), \qquad Z(X) = \prod_{i=1}^{m} (X - \omega^i) \tag{14}$$

se comprueba en un punto aleatorio: si fuera falsa, por el lema de Schwartz–Zippel sólo se cumpliría con probabilidad a lo más $\deg / |\mathbb{F}|$.

Algoritmo 7 Pipeline de un zk-SNARK

4. Setup (una vez por circuito en Groth16; universal en PLONK)
$(pk, vk) \leftarrow \mathsf{Setup}(1^\lambda, C)$ ▷ basta un participante honesto que destruya su aleatoriedad
5. Testigo completo
$z \leftarrow \mathsf{CalcularTestigo}(C,\ \text{públicos} = h,\ \text{privados} = w)$
asegurar $\forall i:\ \langle A_i, z\rangle\langle B_i, z\rangle = \langle C_i, z\rangle$ ▷ si falla, no hay prueba
6. Prueba (costo $O(m \log m)$ por FFT y MSM)
interpolar las restricciones como polinomios $A(X), B(X), C(X)$
$H(X) \leftarrow (A(X)B(X) - C(X))/Z(X)$
comprometer los polinomios (KZG o FRI) y aleatorizar (cegado) ▷ da el conocimiento cero
derivar los retos por Fiat–Shamir sobre la transcripción
$\pi \leftarrow \mathsf{Prove}(pk, h, z)$
7. Verificación (cualquiera, sin w)
devolver $\mathsf{Verify}(vk, h, \pi)$

4.5 Etapa 8 · Integración y auditoría

- **Restricciones faltantes** (*under-constrained circuits*): la causa más frecuente de fallas reales. Toda señal asignada con `<--` en Circom debe quedar restringida con `===`.
- **Aleatoriedad**: nonces y factores de cegado desde un CSPRNG (`secrets` en Python).
- **Entradas públicas**: validarlas dentro del campo y del subgrupo correcto.
- **Separación de dominio** en todos los hashes de Fiat–Shamir.
- **Herramientas de revisión**: Circomspect y Picus para circuitos Circom; pruebas con testigos maliciosos.

5 Librerías en Python y herramientas del ecosistema

Python es excelente para prototipar, enseñar y orquestar, pero ninguna librería nativa de Python para conocimiento cero está auditada para producción. El patrón habitual es prototipar en Python y probar en producción con Rust, Go o Circom, llamados desde Python.

5.1 Librerías de Python

Librería	Qué ofrece	`pip install`	Versión	Estado
py_ecc	secp256k1, alt_bn128 y BLS12-381 con emparejamientos; base para Groth16 o KZG a mano	`py_ecc`	8.0.0 (abr. 2025)	Ethereum Foundation; alfa, no auditada
zksk	protocolos Σ componibles: DLRep, AND, OR, rangos, NI por Fiat–Shamir	`zksk`	0.0.2 (jun. 2020)	EPFL; alfa, para prototipos
tno.zkp. templates	Σ estándar no interactivos y Σ comprimidos (Attema)	`tno.zkp. templates`	0.1.1.post1 (dic. 2023)	TNO PET Lab; Python ≥ 3.8

Librería	Qué ofrece	`pip install`	Versión	Estado
PySNARK	escribir el cómputo en Python y obtener un zk-SNARK; Pinocchio, Groth16 y Bulletproofs	desde GitHub	wiki de 2021	académico, poco activo
zkpy	envoltorio de Circom y snarkjs: Groth16, PLONK y FFLONK	`zkpy`	0.2.0 (may. 2023)	útil para scripts y docencia
ezkl	pruebas de inferencia de modelos ONNX (ZKML) sobre Halo2	`ezkl`	activa	la más madura con API de Python
petlib	envoltorio de OpenSSL para curvas y enteros grandes; base de zksk	`petlib`	0.0.45 (may. 2019)	inactivo

Tabla 4: Librerías de Python para conocimiento cero (consultadas el 27 de octubre de 2026).

Complementos útiles: `hashlib` y `secrets` (biblioteca estándar) para Fiat–Shamir y nonces; `galois` o `sympy` para aritmética de campos finitos y polinomios al enseñar R1CS y QAP.

5.2 Herramientas de producción (fuera de Python)

Herramienta	Tipo	Lenguaje	Desde Python
Circom + snarkjs	circuitos + probador Groth16/PLONK	DSL + JS	`subprocess` o zkpy
Noir (Aztec)	circuitos independientes del backend	DSL tipo Rust	CLI `nargo`
Halo2	PLONKish sin setup confiable	Rust	ezkl o PyO3
arkworks	curvas, Groth16, Marlin	Rust	PyO3
gnark	Groth16 y PLONK de alto rendimiento	Go	CLI o servicio
RISC Zero, SP1, OpenVM	zkVM: se prueba un programa en Rust sin escribir circuitos	Rust	servicio o CLI
Cairo / Stone / Stwo	STARKs de Starknet	Cairo	CLI

Tabla 5: Herramientas de producción y su integración con Python.

Según el recuento de Wavect [15], SP1, RISC Zero y OpenVM 2.0 ya están en producción, mientras que Jolt sigue en alfa. Ruta recomendada para un laboratorio académico: protocolos Σ a mano en Python (sección 6) → circuitos en Circom o Noir orquestados con Python → zkVM cuando el cómputo sea un programa completo.

6 Implementación de referencia en Python

El programa siguiente implementa el algoritmo 3 sobre secp256k1 sin dependencias externas: claves, probador, verificador, simulador y la extracción del secreto cuando se reutiliza el nonce. Fue ejecutado con Python 3 y produce la salida mostrada al final.

Precaución

Es código didáctico: la multiplicación escalar no es de tiempo constante y filtra información por canales laterales.

```python
"""Prueba de conocimiento cero de Schnorr sobre secp256k1 (didáctica, sin dependencias).
No usar en producción: la aritmética no es de tiempo constante."""
import hashlib, secrets

# --- Parámetros de secp256k1 ---
P  = 2**256 - 2**32 - 977
N  = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
Gx = 0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798
Gy = 0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8
G  = (Gx, Gy)

def en_curva(Q):
    if Q is None: return False
    x, y = Q
    return 0 <= x < P and 0 <= y < P and (y*y - x*x*x - 7) % P == 0

def suma(A, B):
    if A is None: return B
    if B is None: return A
    (x1, y1), (x2, y2) = A, B
    if x1 == x2 and (y1 + y2) % P == 0: return None
    if A == B: l = 3*x1*x1 * pow(2*y1, -1, P) % P
    else:      l = (y2 - y1) * pow(x2 - x1, -1, P) % P
    x3 = (l*l - x1 - x2) % P
    return (x3, (l*(x1 - x3) - y1) % P)

def mult(k, Q):
    R = None
    k %= N
    while k:
        if k & 1: R = suma(R, Q)
        Q = suma(Q, Q); k >>= 1
    return R

def neg(Q): return None if Q is None else (Q[0], (-Q[1]) % P)
def ser(Q): return Q[0].to_bytes(32, "big") + Q[1].to_bytes(32, "big")

def reto(Y, T, msg: bytes) -> int:
    """Fiat-Shamir: hash de TODO el contexto público con etiqueta de dominio."""
    h = hashlib.sha256(b"ZK-Schnorr-v1|" + ser(G) + ser(Y) + ser(T) + msg).digest()
    return int.from_bytes(h, "big") % N

def generar_claves():
    x = secrets.randbelow(N - 1) + 1          # testigo secreto
    return x, mult(x, G)                      # (x, Y = x*G)

def probar(x, Y, msg: bytes):
    r = secrets.randbelow(N - 1) + 1          # nonce nuevo en cada prueba
```

```
    T = mult(r, G)
    c = reto(Y, T, msg)
    s = (r + c * x) % N
    return (T, s)

def verificar(Y, msg: bytes, prueba) -> bool:
    T, s = prueba
    if not (en_curva(Y) and en_curva(T)) or not (0 <= s < N):
        return False
    c = reto(Y, T, msg)
    return mult(s, G) == suma(T, mult(c, Y))

def simular(Y):
    """Transcripción interactiva válida sin conocer x."""
    c, s = secrets.randbelow(N), secrets.randbelow(N)
    T = suma(mult(s, G), neg(mult(c, Y)))    # T = s*G - c*Y
    return T, c, s

if __name__ == "__main__":
    x, Y = generar_claves()
    msg = b"login: jose@lab-ia, nonce de sesion 8f3a"
    pi = probar(x, Y, msg)
    print("prueba válida:          ", verificar(Y, msg, pi))
    print("otro mensaje:           ", verificar(Y, b"otro", pi))
    T, s = pi
    print("respuesta alterada:     ", verificar(Y, msg, (T, (s + 1) % N)))

    T, c, s = simular(Y)
    print("simulada verifica:      ", mult(s, G) == suma(T, mult(c, Y)))

    # Error clásico: reutilizar r con dos retos distintos expone x
    r = secrets.randbelow(N); c1, c2 = 11, 29
    s1, s2 = (r + c1*x) % N, (r + c2*x) % N
    x_ext = (s1 - s2) * pow(c1 - c2, -1, N) % N
    print("x extraído por reuso:   ", x_ext == x)
```

Salida al ejecutarlo:

```
prueba válida:          True
otro mensaje:           False
respuesta alterada:     False
simulada verifica:      True
x extraído por reuso:   True
```

Del prototipo a algo utilizable.

1. Sustituir la aritmética propia por una librería de tiempo constante: `coincurve` (libsecp256k1) o Ristretto255 vía libsodium.
2. Derivar el nonce de forma determinista y con sal (estilo RFC 6979 o BIP-340) para que un generador defectuoso no repita r.
3. Serializar puntos comprimidos (33 bytes) y fijar un formato de prueba versionado.

4. Si la afirmación crece (atributos, rangos, hashes), migrar a zksk para Σ compuestos o a un circuito Circom/Noir orquestado con zkpy o `subprocess`.

7 Aplicaciones reales en producción

El conocimiento cero salió del laboratorio por dos vías: privacidad en pagos e identidad, y escalabilidad en cadenas de bloques, donde una prueba corta certifica miles de transacciones. En 2026 se añade la prueba en tiempo real de bloques completos de Ethereum.

Sistema	Sector	Qué se prueba sin revelar	Esquema
Zcash (Sapling, Orchard)	pagos	que una transacción es válida sin mostrar remitente, destinatario ni monto	Groth16; Halo2 en Orchard
Monero	pagos	que los montos ocultos no son negativos	Bulletproofs+
Schnorr / EdDSA, Taproot	firmas	conocimiento de la clave privada	Σ + Fiat–Shamir
ZKsync, Starknet, Scroll, Linea, Polygon zkEVM	rollups	que miles de transacciones fuera de la cadena se ejecutaron bien	PLONK, STARK, envoltura Groth16
SP1, RISC Zero, OpenVM	zkVM	ejecución correcta de un programa, incluida la prueba de bloques de Ethereum	STARK + envoltura SNARK
World ID	identidad	que soy un humano único registrado, sin decir cuál	Semaphore (Groth16)
Google Wallet (Longfellow)	identidad	mayoría de edad a partir de una credencial, sin la fecha de nacimiento	ZK sobre credenciales mdoc
Self, Rarimo, Anon Aadhaar	identidad	edad, nacionalidad o unicidad desde pasaporte o Aadhaar	circuitos sobre firmas oficiales
Filecoin	almacenamiento	que un proveedor guarda las copias que declara	Groth16 (Proof-of-Replication)
Mina	cadena de bloques	el estado completo con una prueba de tamaño constante	pruebas recursivas
EZKL, Lagrange DeepProve	IA verificable	que una salida proviene de cierto modelo sin publicar los pesos	Halo2 y sistemas propios

Tabla 6: Sistemas de conocimiento cero en producción.

Datos de escala reportados por Wavect [15]: el costo promedio de probar un bloque de Ethereum bajó de 1.69 USD a menos de 4 centavos durante 2025; SP1 probó el 99.7 % de los bloques en menos de 12 segundos con 16 GPU RTX 5090; World ID cuenta millones de usuarios verificados.

Precaución

La madurez no elimina los errores: RISC Zero pagó una recompensa de 50 000 USD por una falla de solidez hallada después de auditorías, y Zcash Orchard tuvo una falla de solidez de cuatro años que obligó a la actualización NU6.2 en mayo de 2026 [15].

8 Escenarios donde el conocimiento cero puede usarse

El conocimiento cero encaja cada vez que alguien debe convencer a otro de un hecho sobre datos privados y hoy lo resuelve entregando los datos completos. En cada escenario se indica qué es público y qué queda oculto.

Sector	**Escenario**	**Se prueba (público)**	**Queda oculto**	**Esquema sugerido**
Identidad	Mayoría de edad en tienda o app	edad ≥ 18	fecha de nacimiento, nombre, CURP	rango sobre credencial firmada (BBS+, SNARK)
Identidad	Acceso a edificio o campus	soy miembro vigente	cuál miembro soy	pertenencia a árbol de Merkle (Semaphore)
Autenticación	Inicio de sesión sin contraseña en el servidor	conozco la clave de mi cuenta	la clave	Schnorr NI, OPAQUE
Finanzas	Solicitud de crédito o renta	ingreso mensual $\geq$ umbral	estados de cuenta	rango sobre datos firmados por el banco
Finanzas	Prueba de reservas de un exchange	activos $\geq$ pasivos	saldos individuales	Merkle + pruebas de rango
Finanzas	Cumplimiento antilavado	no figuro en lista de sanciones	identidad y transacciones	no pertenencia a un conjunto
Impuestos	Declaración ante una autoridad	el impuesto sigue la regla fiscal	facturas una a una	SNARK sobre el cálculo
Salud	Requisito para ensayo clínico	cumplo criterios de inclusión	diagnóstico e historial	circuito sobre expediente firmado
Salud	Certificado de vacunación o aptitud	certificado válido y vigente	resto del expediente	divulgación selectiva
Educación	Verificación de título o calificaciones	promedio ≥ 9 o título válido	boleta completa	firma institucional + rango
Educación	Exámenes en línea	respuesta correcta antes del cierre	la respuesta	compromiso + revelación diferida
Votación	Elecciones electrónicas verificables	voto válido (0 o 1) y contado	por quién voté	pruebas OR + cifrado homomórfico
Suministro	Origen de un producto	lote certificado y conforme a norma	proveedores, precios, volúmenes	SNARK sobre registros firmados
IA	Inferencia verificable	la salida la produjo el modelo M	pesos o entrada	ZKML (EZKL, Halo2)
IA	Auditoría de entrenamiento	no se usó un conjunto prohibido	conjunto de entrenamiento	zkVM (aún costoso)
Ciberseguridad	Divulgación de vulnerabilidades	tengo un exploit contra el binario B	el exploit	zkVM sobre la ejecución
Ciberseguridad	Contraseña filtrada	mi contraseña no está en la base filtrada	la contraseña	PSI o no pertenencia
Geolocalización	Servicios por zona	estoy dentro de la región R	coordenadas exactas	rango sobre coordenadas firmadas
Juegos	Juego justo	resultado generado con las reglas	la semilla hasta el final	compromiso + SNARK
Gobierno	Padrones de apoyos sociales	beneficiario elegible, sin cobro doble	identidad	nulificadores (Semaphore)
Matemáticas	Poseer una solución	conozco una 3-coloración, un Sudoku o un factor de n	la solución	GMW [3] o SNARK

Tabla 7: Escenarios de uso del conocimiento cero.

Cuatro preguntas antes de elegir un escenario

1. ¿Quién firma los datos de origen? Sin una firma confiable, el probador puede mentir sobre su entrada.
2. ¿Cuánto puede tardar la prueba en el dispositivo del usuario?
3. ¿Dónde se verifica: servidor, teléfono o contrato inteligente?
4. ¿Debe el sistema resistir computadoras cuánticas?

Referencias

[1] S. Goldwasser, S. Micali y C. Rackoff. *The Knowledge Complexity of Interactive Proof-Systems*. STOC, 1985.

[2] A. Fiat y A. Shamir. *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*. CRYPTO, 1986.

[3] O. Goldreich, S. Micali y A. Wigderson. *Proofs that Yield Nothing but Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems*. Journal of the ACM 38(3), 1991.

[4] R. M. Karp. *Reducibility Among Combinatorial Problems*. En *Complexity of Computer Computations*, Plenum, 1972.

[5] M. Blum. *How to Prove a Theorem So No One Else Can Claim It*. Proceedings of the International Congress of Mathematicians, 1986.

[6] O. Goldreich y H. Krawczyk. *On the Composition of Zero-Knowledge Proof Systems*. SIAM Journal on Computing 25(1), 1996.

[7] C. P. Schnorr. *Efficient Signature Generation by Smart Cards*. Journal of Cryptology 4(3), 1991.

[8] R. Cramer, I. Damgård y B. Schoenmakers. *Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols*. CRYPTO, 1994.

[9] J. Groth. *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT, 2016.

[10] A. Gabizon, Z. Williamson y O. Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. IACR ePrint 2019/953.

[11] E. Ben-Sasson, I. Bentov, Y. Horesh y M. Riabzev. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. IACR ePrint 2018/046.

[12] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille y G. Maxwell. *Bulletproofs: Short Proofs for Confidential Transactions and More*. IEEE S&P, 2018.

[13] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.

[14] D. Boneh y V. Shoup. *A Graduate Course in Applied Cryptography*, caps. 19–20.

[15] Wavect. *Zero-Knowledge Proofs in Production 2026*. https://wavect.io/blog/zero-knowledge-proofs-production-2026/. Consultado el 27 de octubre de 2026.